OBJECT-ORIENTED PROGRAMMING

Jamaican Football League and Match-Day Management System

Six-Week Group Project Specification

Project duration	Group size	Recommended platform
6 weeks	3-5 students	C++ or Java

Student Project Brief

Academic Year: _____
Lecturer: _____
Submission Date: _____

1. Project overview

The Kingston Premier Football League (KPFL) requires a group of three to five members to create an object-oriented solution to manage their football competition. The KPFL consists of football teams, players, coaches, match officials, stadiums, fixtures, ticket sales, merchandise, and match-day operations.

The minimum submission should be a functional application with persistent data storage using sequential access files, random access files, or both. You may include a graphical user interface (GUI) as the interface for your solution; however, it should only complement the functionality of your solution, not replace the required object-oriented design, validation, exception handling, and testing.

2. Scenario

The scenario is as follows: In the Parish of Kingston, the Kingston Premier Football League (KPFL) runs a data-driven football competition each year. The competition organizers allow all teams to request permission to use a stadium for practice before each match. Each Team has a name, parish, registration number, squad size, league status, players, and coaching staff.

Each Stadium has a stadium ID, name, location, seating capacity, field condition, ticket price, and a record of the teams assigned to the current fixture. The Football League manages teams, stadiums, fixtures, results, and league standings. A League Commissioner approves registrations, inspects match preparations, and coordinates the assignment of Match Officials to each match.

The League Commissioner, Player, Coach, Match Official, and Spectator are all specialised types of Person. Each Person has an identification number, name, age, and gender. On match days, spectators may purchase tickets and products from the Stadium Shop, including merchandise, food, and beverages. All prices include a 15% general consumption tax (GCT).

3. Learning outcomes

Upon successful completion of this group project, students should be able to:

- analyze a real-world problem and translate its requirements into an object-oriented model;
- appropriately apply encapsulation, inheritance, abstraction, and polymorphism;
- implement associations, aggregations, and compositions among collaborating classes;
- effectively use exception handling and persistent storage;
- design and execute tests for normal, boundary, and invalid cases; and
- collaborate responsibly while explaining and defending their individual contributions.

4. Core functional requirements

1. Register, update, search for, and display football teams.
2. Add, update, and remove players and coaches associated with a team.
3. Register stadiums and record seating capacity and field condition.
4. Create fixtures and assign teams, stadiums, dates, and times.
5. Register and assign match officials without scheduling conflicts.
6. Record match results and calculate league points and standings.
7. Sell tickets while monitoring stadium capacity.
8. Manage products and complete stadium-shop transactions.
9. Calculate subtotal, tax, total payment, and change.
10. Save and reload system data using files.
11. Produce at least three useful management reports.
12. Use exceptions to handle invalid data and prohibited operations.

5. Mandatory Business Rules for the competition

- An unregistered or suspended team cannot be assigned to a fixture.
- A team cannot participate in two fixtures scheduled for the same day and time.
- A match official cannot be assigned to conflicting fixtures.
- Ticket sales cannot exceed the stadium's seating capacity.
- Match scores and quantities cannot be negative.
- Players must satisfy the league's stated minimum-age rule.
- A transaction cannot be completed when payment is insufficient.
- A fixture cannot be completed until a valid result has been recorded.

6. Recommended object-oriented model

Class	Primary responsibility
Person	Abstract superclass containing common personal information.
Player	Stores position, jersey number, goals, and playing status.
Coach	Stores qualification, coaching role, and team assignment.
LeagueCommissioner	Approves teams and coordinates league operations.
MatchOfficial	Stores officiating role, rank, and availability.
FootballTeam	Manages players, coaches, points, and registration information.
Stadium	Manages capacity, location, field condition, and occupancy.
Fixture	Stores teams, stadium, date, time, and fixture status.
MatchResult	Records scores, winner, and match statistics.
Ticket	Represents a match ticket, category, and price.
Spectator	Represents a person purchasing tickets or products.
Product	Abstract superclass for items sold by the stadium shop.
Merchandise	Represents jerseys, flags, and related products.
FoodItem	Represents food and beverage products.
StadiumShop	Manages inventory, sales, tax, and receipts.
FootballLeague	Coordinates teams, fixtures, stadiums, and standings.

Groups may modify this model where the alternative is consistent, clearly documented, and justified using object-oriented design principles.

7. Required OOP and programming features

Feature	Minimum expectation
Encapsulation	Private/protected data with controlled access and validation.
Inheritance	A meaningful Person hierarchy and at least one additional hierarchy.
Polymorphism	Overridden methods called through superclass or interface references.
Abstraction	At least one abstract class or interface.
Relationships	Use association, aggregation, and composition appropriately.
Exceptions	Custom exceptions for important business-rule violations.
Persistence	Save and reload application data using files (Sequential, Random Access, or Both).
Testing	Documented tests covering normal, boundary, and invalid inputs.

8. Five-week development schedule

Week	Focus	Checkpoint deliverable
1	Requirements and planning, Object-oriented design	Requirements, use cases, roles, task allocation, and project schedule. UML class diagram, relationships, interface sketches, storage design, and test plan.
2	Core implementation	Person hierarchy, teams, personnel, stadiums and initial unit tests.
3	League operations	Fixtures, officials, results, standings, collections and exceptions.
4	Match-day services	Ticketing, shopping, tax, persistence, searching and reports.
5	Integration and defense	Completed system, testing report, demonstration, technical report, and reflection.

9. Weekly requirements

Week 1 - Requirements analysis and planning and System Design

- Identify functional and non-functional requirements.
- Develop at least five use cases.
- Identify actors, constraints, and business rules.
- Assign initial group responsibilities.
- Create a shared repository/workspace and project schedule.
- Submit a UML class diagram with multiplicities and relationship types.
- Explain inheritance, composition, and aggregation decisions.
- Design the menu or graphical interface.
- Define the persistence approach and validation strategy.
- Prepare the initial test plan.

Week 2 - Core implementation

- Implement constructors, accessors, mutators, and validation.
- Implement the Person hierarchy, teams, coaches, players, and stadiums.
- Implement suitable collections and object display methods.
- Demonstrate a working console prototype and initial tests.

Week 3 - League operations

- Implement fixtures, stadium assignment, and official assignment.
- Record match results and update standings.
- Apply the points system: win = 3, draw = 1, loss = 0.
- Implement and demonstrate custom exceptions.

Week 4 - Match-day services and persistence

- Implement ticket sales and capacity monitoring.
- Implement inventory, shop transactions, tax, and receipts.
- Save and reload data.
- Implement search facilities and at least three reports.

Week 5 - Integration, testing and presentation

- Integrate all modules and correct defects.
- Test normal, boundary, and invalid cases.
- Submit code, documentation, contribution log, and reflections.
- Deliver a 10–15-minute demonstration and answer individual questions in interviews.

10. Suggested group roles

Group size	Suggested primary responsibilities
3 students	Project lead and league operations; team and personnel; sales, persistence, and testing.
4 students	Project/integration lead; team and personnel; fixtures and stadiums; sales, persistence, and quality assurance.
5 students	Project/integration lead; team and personnel; fixtures and stadiums; ticketing and shop; data, testing, and documentation.

Roles indicate each member's primary responsibility, but everyone is encouraged to contribute to coding, testing, documentation, and the final presentation. When helpful, members can rotate tasks or pair up to build shared understanding.

11. Final submission package

- Requirements and design document
- UML class diagram
- Complete, organized source code
- Executable application and setup instructions
- Test plan with results and evidence
- User guide
- Technical report
- Contribution log
- Individual reflection from every member
- Presentation slides and live demonstration

12. Assessment rubric

Component	Weight	Assessment emphasis
Requirements analysis and planning	10%	Accurate requirements, feasible scope, use cases, plan and role allocation.
UML and object-oriented design	15%	Coherent classes, responsibilities, relationships, multiplicities and justified design.
OOP implementation	25%	Effective encapsulation, inheritance, abstraction, polymorphism and collaboration among objects.
Functionality and business rules	15%	Reliable implementation of the required operations and constraints.
Exceptions, persistence and testing	15%	Robust error handling, reliable storage and meaningful test coverage.
Code quality and documentation	10%	Readable, modular, maintainable code and clear documentation.
Presentation and demonstration	5%	Professional demonstration, clear communication, and an operational system.
Individual contribution and defense	5%	Evidence of contribution and ability to explain design and code.
Total	100%	

13. Individual accountability

The submission is assessed as a group product, but individual marks may be adjusted using the following evidence:

- weekly contribution logs and repository history;
- peer evaluations;
- attendance and participation in weekly demonstrations;
- individual reflections;
- questions directed to specific students during the final defense; and
- each member's ability to explain the design and code attributed to them.

A group member who cannot explain the claimed contribution will not automatically receive the group's full implementation mark.

14. Submission and presentation requirements

- Submit one compressed project folder using the naming convention specified by the lecturer.
- Include clear instructions for compiling, configuring, and running the application.
- Do not submit generated build folders unless requested.
- The demonstration must use prepared sample data but should also accept live input.
- Every group member must speak and answer at least one technical question.
- Sources, frameworks, and AI-assisted work must be acknowledged according to course policy.

15. Optional extension features

Attempt extensions only after you have completed and tested all core requirements.

- JavaFX or another graphical user interface
- database-backed persistence
- player-transfer management
- disciplinary cards and suspension tracking
- top-scorer and attendance analytics
- role-based login and access control
- exportable PDF or CSV reports

16. Lecturer approval checkpoint

Item	Status / comments
Scope approved	
UML approved	
Week 3 prototype checked	
Week 4 operations checked	
Week 5 integration checked	
Final submission received	

End of Group Project